flutter_invoice_generator

Learning Guide - how the package is designed and built, phase by phase, in simple language

Chapter 1: Phase 1 - Architecture Plan

Status: waiting for your approval before writing code.

1. What are we building?

We are building a **Flutter package** (also called a library or SDK). A package is not an app. Nobody 'opens' it on their phone. Instead, other Flutter developers add it to their own apps with one line in `pubspec.yaml`, and then they get ready-made classes and functions they can call.

Our package gives them one job, done very well: **turn invoice data into a professional PDF**, then let them preview, save, share or print it.

> **In simple words**
> Think of the package as a machine. Developers put in data (who sells, who buys, what items, which tax) and the machine gives back a finished PDF invoice. They never need to know how the machine works inside.

The developer experience we want

```
final invoice = Invoice(
  invoiceNumber: 'INV-1001',
  issueDate: DateTime.now(),
  dueDate: DateTime.now().add(const Duration(days: 30)),
  seller: Business(name: 'ABC Technologies', email: 'hello@abc.com'),
  customer: Customer(name: 'John Doe', email: 'john@example.com'),
  items: [
    InvoiceItem(name: 'Flutter App Development', quantity: 1, unitPrice: 500),
    InvoiceItem(name: 'UI/UX Design', quantity: 2, unitPrice: 100),
  ],
  tax: Tax(name: 'VAT', percentage: 13),
  discount: Discount.percentage(10),
  currency: Currency.usd,
);

final pdf = await InvoiceGenerator.generate(invoice);  // build the PDF
await InvoiceGenerator.share(invoice);                 // share sheet
await InvoiceGenerator.print(invoice);                 // print dialog
await InvoiceGenerator.save(invoice);                  // save to a file
```

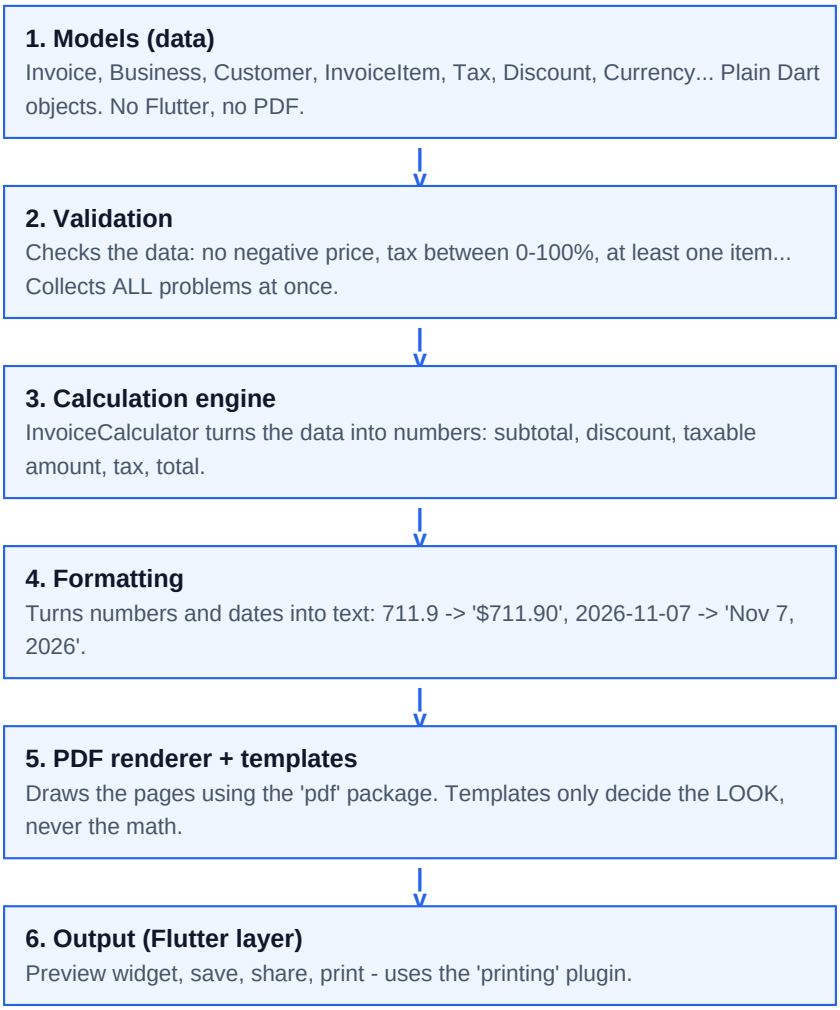
Important: your current folder

Right now `pdf_invoice/` was created with `flutter create` as an **app**. It has `android/ ios/ web/ macos/ windows/ linux/` folders and a `main.dart`. A package does not need those at the root. In Phase 2 we convert it:

- Remove the platform folders and `lib/main.dart` from the root (they are only generated boilerplate, no code of yours).
- Rename the package to `flutter_invoice_generator` in `pubspec.yaml`.
- Create a separate demo app inside `example/`. That demo app is where the platform folders belong.

2. The big picture: layers

The most important idea in this package is **separation**. Each layer does one job and only talks to the layer next to it. This keeps the code easy to test and easy to change later.



> **In simple words**
>
> It is like a restaurant. Models are the ingredients, validation is checking the ingredients are fresh, the calculator is the recipe, templates are how the plate is decorated, and output is the waiter delivering the plate. The decorator never changes the recipe.

Why this matters

- **Testing:** we can test the money math with no PDF at all - fast and exact.
- **Templates:** Classic, Modern and Minimal share the same engine. Only the drawing code differs, so a bug fixed in the math is fixed for all.
- **Server use:** layers 1-5 do not need Flutter UI, so the same code could later generate invoices on a Dart server.

3. Folder structure

This follows the structure you asked for, with a few small additions (marked NEW) that I explain below.

```
flutter_invoice_generator/
|-- lib/
|   |-- flutter_invoice_generator.dart   <- the ONLY public entry file
|   `-- src/                             <- private implementation
|       |-- models/
|       |   |-- invoice.dart
|       |   |-- invoice_item.dart
|       |   |-- business.dart
|       |   |-- customer.dart
|       |   |-- tax.dart
|       |   |-- discount.dart
|       |   |-- currency.dart
|       |   |-- payment_information.dart
|       |   `-- invoice_logo.dart          NEW: bytes / asset / network logo
|       |-- calculation/
|       |   |-- invoice_calculator.dart
|       |   |-- invoice_totals.dart
|       |   `-- money.dart                 NEW (internal): exact money math
|       |-- validation/                    NEW
|       |   `-- invoice_validator.dart
|       |-- formatting/                    NEW
|       |   `-- invoice_formatter.dart     dates + money text
|       |-- pdf/
|       |   |-- invoice_generator.dart     public facade (generate/save/...)
|       |   |-- invoice_pdf.dart           the result object
|       |   |-- invoice_pdf_builder.dart   internal engine (MultiPage)
|       |   |-- invoice_theme.dart         colors, fonts, sizes
|       |   |-- invoice_labels.dart        NEW: 'INVOICE', 'Bill To'... (translatable)
|       |   |-- sections/                  NEW: shared building blocks
|       |   |   (header, parties, items_table, summary, footer)
|       |   `-- templates/
|       |       |-- invoice_template.dart
|       |       |-- classic_template.dart
|       |       |-- modern_template.dart
|       |       `-- minimal_template.dart
|       |-- preview/
|       |   `-- invoice_preview.dart
|       |-- output/                        NEW: save / share / print
|       |   `-- invoice_output.dart
|       `-- exceptions/
|           `-- invoice_exceptions.dart
|-- assets/fonts/                         NEW: bundled Unicode font
|-- example/                             demo Flutter app
|-- test/  (models/ calculation/ validation/ pdf/)
|-- docs/learning/                       this guide (not published)
|-- README.md  CHANGELOG.md  LICENSE
|-- pubspec.yaml  analysis_options.yaml
```

Why lib/src?

In Dart, anything under `lib/src/` is considered private. Developers only import `package:flutter_invoice_generator/flutter_invoice_generator.dart`, and that file decides exactly what is visible using `export`. If a class is not exported, we can rename or rewrite it later without breaking anyone's app.

In simple words

lib/src is the kitchen. The export file is the menu. Customers only order from the menu, so we can reorganise the kitchen any time.

4. Dependencies (and why each one)

I checked pub.dev on 7 Nov 2026 for the current versions. Fewer dependencies means fewer version conflicts for the people who install us.

Package	Version	Why we need it	Could we avoid it?
pdf	^3.13.1	Builds the PDF: pages, tables, text, images, MultiPage for long invoices. Also includes 'barcode', so QR codes are free.	No - this is the core.
printing	^5.15.1	Print dialog, share sheet, PDF preview widget, loading network images. Supports Android, iOS, macOS, Windows, Linux, Web.	No - writing native print code for 6 platforms is huge.
intl	>=0.19.0 <0.21.0	Date formats ('07/11/2026') and number grouping ('1,234.50') for any locale.	Possible, but locale rules are hard. A WIDE range is used because Flutter's own localization pins one intl version.
path_provider	^2.x (to confirm)	Only for save(): finds the app documents folder on each platform.	Yes - see decision below.
meta	(already via Flutter)	@immutable annotations.	Free.

Decision for you to approve

save(): Option A (recommended): use path_provider so `save(invoice)` works with no arguments and returns the saved file path. Option B: no extra dependency, but the developer must pass a directory: `save(invoice, directory: dir)`. On Web, save() downloads the file through the browser either way.

A hidden problem: currency symbols and fonts

PDFs need a font file that contains every character we print. The built-in PDF font (Helvetica) does **not** contain the Indian Rupee sign or many non-English letters. A Nepali shop printing a customer name in Devanagari would get empty boxes.

Decision for you to approve

Bundle one open-licence font (Noto Sans, regular + bold, about 1 MB) as the default, which includes symbols like the Rupee sign. Developers can still pass any font through InvoiceTheme. This keeps generation **fully offline** (no downloading Google Fonts at runtime).

5. The models (the data)

All models are **immutable**: once created they never change. To 'edit', you call `copyWith(...)` which returns a new copy. This prevents bugs where one screen changes an invoice that another screen is still printing.

```
Invoice
 |-- invoiceNumber, issueDate, dueDate
 |-- seller   : Business   (name, email, phone, address, website,
 |                          taxId, registrationNumber, logo)
 |-- customer : Customer   (name, companyName, email, phone, address, taxId)
 |-- items    : List<InvoiceItem>
 |                (name, description, quantity (double), unitPrice,
 |                 sku, unit, tax?, discount?)
 |-- currency : Currency    (code, symbol, decimalDigits, symbolOnLeft)
 |-- taxes    : List<Tax>   (name, percentage)  <- 'tax:' shortcut for one
 |-- discount : Discount?   (percentage or fixed)
 |-- paymentInformation : PaymentInformation? (all fields optional)
 |-- notes, terms
 |-- details  : List<InvoiceDetail>  (extra rows shown, e.g. 'PO Number')
 `-- metadata : Map<String, String>  (your own data, never printed)
```

Design choices explained

- **Quantity is a double**, so 2.5 hours or 0.75 kg work.
- **Multiple taxes:** you can write `tax: Tax(...)` for the simple case or `taxes: [vat, serviceTax]`. No country rules are hard-coded - we only use the percentage you give.
- **Discount** gets friendly constructors: `Discount.percentage(10)` and `Discount.fixed(50)`. The long form `Discount(type: DiscountType.percentage, value: 10)` also works.
- **Currency** has ready constants (usd, eur, gbp, inr, npr, aud, cad, jpy) and you can make your own: `Currency(code: 'NPR', symbol: 'Rs.', decimalDigits: 2)`. JPY uses 0 decimals.
- **Logo** can come from bytes, an asset or (only if you ask) the network.
- **Optional fields everywhere** - adding a new optional field in v1.1 does not break anyone. That is how we stay backward compatible.

> **Decision for you to approve**
>
> You listed 'template configuration' as an Invoice field. I recommend **not** putting it inside Invoice. An invoice is data (what was sold); the template is presentation (how it looks). Instead: `InvoiceGenerator.generate(invoice, template: InvoiceTemplate.modern, theme: myTheme)`. Then the same invoice can be printed in any style, and saving invoices to a database never stores colours.

6. The calculation engine

Money math is where invoice software most often goes wrong. Computers store decimals like 0.1 in binary and cannot store them exactly:

```
print(0.1 + 0.2);      // 0.30000000000000004   (not 0.3!)
print(1.005 * 100);    // 100.49999999999999    (should be 100.5)
```

Our strategy: count in the smallest unit

Internally we never add doubles. We convert every amount into **whole minor units** (cents for USD, paisa for NPR, yen for JPY) stored as exact integers, do the math, and convert back only for display. We read the number through its shortest text form (1.005 stays '1.005'), so the binary error never enters the calculation. This needs **no extra package**.

> **In simple words**
> Instead of saying 'one dollar and five cents' (1.05) we say '105 cents'. Whole numbers never have rounding surprises.

The formula (worked example)

Step	Rule	Example
Line total	quantity x unitPrice - item discount	1 x 500 = 500.00 ; 2 x 100 = 200.00
Subtotal	sum of line totals	700.00
Discount	invoice discount on the subtotal	10% of 700 = 70.00
Taxable amount	subtotal - discount	630.00
Tax	each tax % of the taxable amount	VAT 13% of 630 = 81.90
Total	taxable amount + all taxes	711.90

Rounding rules (documented in code and README)

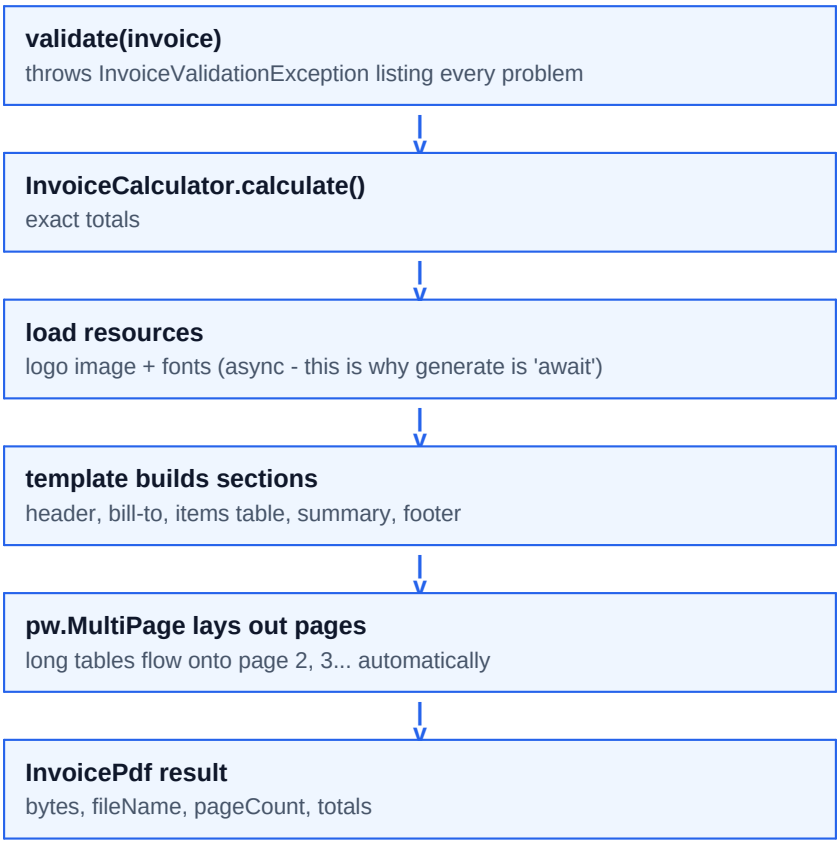
- Rounding happens to the currency's decimal places (2 for USD, 0 for JPY).
- Mode: **round half away from zero** (0.125 -> 0.13), the normal commercial rule.
- We round each line total, the discount and each tax amount once. The total is then an exact sum, so the numbers on the PDF always add up.
- If an item has its own tax, that item is taxed with its own rate; the invoice discount is shared across items in proportion to their value (any leftover cent goes to the last item, so nothing is lost).

The result is an immutable `InvoiceTotals` object: `subtotal, discountAmount, taxableAmount, taxLines (one per tax), taxAmount, total, lines`. Developers can use it without making a PDF at all:

```
final totals = InvoiceCalculator.calculate(invoice);
print(totals.total); // 711.9
```

7. PDF generation and templates

What generate() does



What you get back

```
final pdf = await InvoiceGenerator.generate(invoice);
pdf.bytes;      // Uint8List - upload it, email it, store it
pdf.fileName;   // 'INV-1001.pdf'
pdf.pageCount;  // 3
pdf.totals;     // the same numbers printed on the PDF
```

Returning our own small `InvoicePdf` class (instead of raw bytes) means we can add fields later without breaking anyone.

Multiple pages

The pdf package has a widget called `MultiPage`. We give it a long list of content; it fills page 1, then continues on page 2. The items table header row repeats on each new page, the big business header is drawn only on page 1, and every page gets 'Page 2 of 3' in the footer. We will test 1, 10, 50 and 150 items and check the real page count.

Templates without copy-paste

Every invoice has the same five **sections**. Shared section builders live in `pdf/sections/`. A template is a small class that decides how each section looks (colours, alignment, header band or not), reusing those builders.

Template	Look
Classic	Traditional: logo left, 'INVOICE' right, bordered table, grey header row.
Modern	SaaS style: coloured header band, rounded summary card, zebra rows.
Minimal	Lots of white space, thin lines, no fills, small type.

> **Decision for you to approve**
>
> In v1, `InvoiceTemplate` exposes only the three built-in templates (`InvoiceTemplate.classic`, etc.). Opening it up for fully custom templates is planned, but doing it in v1 would freeze our internal drawing code as public API. Keeping it closed now lets us improve it freely.

Customization: InvoiceTheme and InvoiceLabels

```
InvoiceTheme(
  primaryColor: PdfColor.fromHex('#2563EB'),
  secondaryColor: PdfColors.grey700,
  font: myFont, boldFont: myBoldFont,
  baseFontSize: 10, spacing: 12, borderRadius: 6,
  pageFormat: PdfPageFormat.a4,           // or PdfPageFormat.letter
  dateFormat: 'dd/MM/yyyy', locale: 'en_US',
)
InvoiceLabels(invoiceTitle: 'TAX INVOICE', billTo: 'Bill To', ...)
```

Labels are separate so a Nepali or French developer can translate every word on the PDF without touching colours.

8. Preview, save, share, print

Preview widget

`InvoicePreview(invoice: invoice)` is a Flutter widget. Two ways to build it:

Option	Good	Bad
A. Show the real PDF (recommended)	Looks exactly like the PDF, always. Little code. Uses printing's PdfPreview.	Slightly slower to appear.
B. Re-draw it with Flutter widgets	Instant, scrollable.	Two layouts to keep in sync - they WILL drift apart over time.

Save / share / print

Platform	Print	Share	Save
Android	yes	yes (share sheet)	yes (app documents)
iOS	yes	yes (share sheet)	yes (app documents)
macOS	yes	yes	yes (needs sandbox file permission)
Windows	yes	save dialog	yes
Linux	yes	save dialog	yes
Web	browser print	browser download	browser download

This table is from the printing package docs. In Phase 7 I will verify what I can on your Mac and clearly mark anything not tested.

Error handling

```
InvoiceException                    (base - catch everything)
 |-- InvoiceValidationException     bad data. Has .issues: a list like
 |                                  ['items[2].quantity must be >= 0',
 |                                   'tax VAT: percentage must be 0-100']
 |-- InvoiceImageException          logo missing / broken / unsupported
 `-- InvoiceGenerationException     anything else while building the PDF
```

Validation collects **all** problems at once, so a developer fixes them in one go instead of one crash at a time. Rules: invoice number not empty, at least one item, quantity >= 0, price >= 0, tax 0-100%, percentage discount 0-100%, fixed discount not bigger than the subtotal, due date not before issue date.

QR codes

Not mandatory in v1. Because 'pdf' already includes the barcode library, an optional `InvoiceQrCode` (payment URL, invoice URL or custom text) costs zero new dependencies. I reserve an optional field for it and plan it for Phase 5 as a small extra.

9. The plan, phase by phase

Phase	What	You push after?
1	Architecture plan + this guide (now)	Yes - first commit
2	Convert to package, models, exports, model tests	Yes
3	Calculation engine + many tests	Yes
4	PDF generator + Classic template + multi-page tests	Yes
5	Theme, labels, Modern + Minimal templates, (QR)	Yes
6	InvoicePreview widget	Yes
7	save / share / print + platform table	Yes
8	Example app	Yes
9	README, CHANGELOG, API docs	Yes
10	pub.dev dry-run, fix everything	Yes - then publish

After every phase I stop, tell you what I did and how, add a chapter to this guide, and give you a commit message. I never commit or push - you do.

How to push (first time)

```
cd /Users/prajan/Desktop/package/pdf_invoice
git init
git add .
git commit -m "docs: add architecture plan and learning guide (phase 1)"
git branch -M main
git remote add origin https://github.com/<you>/flutter_invoice_generator.git
git push -u origin main
```

Future pro features (not in the free core)

Extra premium templates, advanced branding, custom sections, recurring invoices. The core stays a clean, free SDK - no payments or licences inside the package.